

Simpler and More Efficient Rank Estimation for Side-Channel Security Assessment

Cezary Glowacz¹, Vincent Grosso², Romain Poussier², Joachim Schüth¹,
and François-Xavier Standaert²(✉)

¹ Security Consulting and Engineering, T-Systems GEI GmbH, Bonn, Germany

² ICTEAM/ELEN/Crypto Group, Université catholique de Louvain,
Louvain-la-neuve, Belgium
`fstandae@uclouvain.be`

Abstract. Rank estimation algorithms allow analyzing the computational security of cryptographic keys for which adversaries have obtained partial information thanks to leakage or cryptanalysis. They are particularly useful in side-channel security evaluations, where the key is known by the evaluator but not reachable with exhaustive search. A first instance of such algorithms has been proposed at Eurocrypt 2013. In this paper, we propose a new tool for rank estimation that is conceptually simpler and much more efficient than this previous proposal. It allows approximating the key rank of (128-bit, 256-bit) symmetric keys with very tight bounds (i.e. with less than one bit of error), almost instantaneously and with limited memory. It also scales nicely to larger (e.g. 1024-bit) key sizes, for which the previous algorithm was hardly applicable.

1 Introduction

Despite progresses in the analysis and understanding of side-channel attacks, empirical evaluations remain an essential ingredient in the security assessment of leaking devices. The main reason for this fact is that the leakage of cryptographic implementations is highly device-specific. This implies that the actual security level provided by ad hoc countermeasures such as masking (e.g. [2, 13] and related works) or shuffling (e.g. [7, 19] and related works) may depend on the underlying technology on which they are running (e.g. glitches in integrated circuits are an illustration of this concern [11]). In fact, even in leakage-resilient primitives that aim to prevent/mitigate side-channel attacks by cryptographic design, the need to bound/quantify the leakages in a rigorous way is an important ingredient for connecting formal analysis with concrete security levels (e.g. [5, 16]).

In this context, the usual strategy for an evaluation laboratory is to launch a set of popular attacks, and to determine whether the adversary can break the implementation (i.e. recover the key). The vast majority of these popular attacks are “divide-and-conquer” ones¹, where different pieces of a master key

¹ Including but not limited to Kocher et al.’s seminal Differential Power Analysis (DPA) [8], Brier et al.’s Correlation Power Analysis (CPA) [1], Chari et al.’s Template Attacks (TA) [3], Gierlichs et al.’s Mutual Information Analysis (MIA) [6] and Schindler et al.’s stochastic approach based on Linear Regression (LR) [14]. Following [10], we will use the term “standard DPAs” for those attacks.

are recovered independently, and then recombined via enumeration [12, 17]. But as recently observed by Veyrat-Charvillon, Gérard and Standaert at Eurocrypt 2013, such security evaluations are limited to the computational power of the evaluator [18]. This is typically a worrying situation since it sets a hard limit to the decision whether an implementation is “practically secure”. For example, one could decide we have a practically secure AES implementation as soon as the number of keys to enumerate is beyond 2^{50} , but this does not provide any hint whether the concrete security level is 2^{51} or 2^{120} . The latter makes a significant difference in practice, especially in view of the possibility of improved measurement setups, signal processing, information extraction, . . . , that usually has to be taken into account for any physical security evaluation, e.g. via larger security margins. As a consequence, the main contribution in [18] was to introduce a rank estimation algorithm which enables evaluators to (quite efficiently) approximate the security level of any implementation, by approximating the position of the master key in the list of 2^{128} possible ones provided by an attack (even if it is beyond enumeration power). This allowed, for the first time, to compute all the security metrics introduced in [15] and to summarize them into “security graphs” (i.e. plots of the adversary’s success probability in function of the number of side-channel measurement and enumeration power).

Technically, the Eurocrypt 2013 algorithm essentially results from the time vs. memory tradeoff between depth-first and breadth-first search in a large data structure representing the key space. More precisely, since depth-first exploration of the key space is too computationally intensive, it rather exploits breadth-first search up to the memory limits of the computing device on which rank estimation is performed. This allows the algorithm to rapidly converge towards reasonably accurate bounds on the key rank. But of course, it implies that refining the bounds becomes exponentially difficult at some point, which may lead to limited accuracies in certain contexts (e.g. large key sizes, typically). Concretely, the representation of a side-channel attack’s results also has a strong impact on the efficiency of the Eurocrypt 2013 rank estimation. For example in the AES case, representing a DPA outcome as 8 lists of size 2^{16} leads to more (time) efficient rank estimation than representing it as 16 lists of size 2^8 . Using a (more memory consuming) representation with 5 lists of 2^{24} elements and one list of 2^8 elements typically allowed bounds with approximately 10 bits of tightness² within seconds of computation, and bounds with approximately 5 bits of tightness within minutes of computation, for a 128-bit key leading to a post side-channel attack security level of 80 bits. Note that the time complexity of the latter rank estimation algorithm is dependent of the estimated security level (and 80-bit was the experimental worst-case in the 128-bit example of [18]). Summarizing, the Eurocrypt 2013 algorithm provides satisfying estimations of the key rank as long as the key size is limited (to symmetric key sizes, typically) and the tightness required by the evaluators can be left to a couple of bits.

In this paper, we provide an alternative rank estimation algorithm that enjoys simplicity and (much) improved (time and memory) efficiency. The algorithm

² Measured with the log of the ratio between the upper and lower bounds.

essentially works in four steps. First, we express the DPA outcome with lists of log probabilities (each list corresponding to a piece of key). Second, we compute the histograms of these log probabilities for all the lists, with a sufficient number of equally-sized bins. Third, we recursively compute the convolution between these histograms. Eventually, we approximate the security level from the last histogram as the number of keys having larger log probabilities than the correct one (that is known by the evaluator). Bounds can additionally be obtained by tracking the quantization errors (depending on the bins' width). Besides its simplicity, this algorithm leads to bounds with less than one bit of tightness within seconds of computation (using the same computing platform as for the previous estimates). Furthermore, and contrary to the Eurocrypt 2013 algorithm, it nicely scales to larger key sizes and leads to rank estimations with good tightness for key sizes up to 1024-bit in our experiments (probably larger if needed).

We finally recall that the proposed algorithm is not limited to physical security evaluations, and is potentially useful in any cryptanalysis context where experiments are needed to validate an hypothetical attack model as well.

Related Works. In [9], Lange et al. adapt the problem of enumeration and rank estimation to the asymmetric setting, where the pieces of key are typically not independent. Ye et al. propose an alternative approach to quantify the computational security left after a side-channel attack, based on the (probabilistic) leakage models derived during such attacks [21]. It can potentially be applied without knowledge of the master key, which comes at the cost of possible biases if the models are imperfect. Eventually, the recent work in [4] describes a solution to bound the security of a leaking implementation based on (simpler to estimate) information theoretic metrics, which borrows from our following results.

2 Background

2.1 Side-Channel Cryptanalysis

Details on how divide-and-conquer side-channel attacks actually extract information about the master key are not necessary for describing the rank estimation problem. For the rest of the paper, we only need to specify the DPA outcomes as follows. Say we target an n -bit master key k and cut it into $N_p = \frac{n}{b}$ pieces of b bits, next denoted as subkeys k_i (for simplicity, we assume that b divides n). The side-channel adversary uses the leakages corresponding to a set of q inputs $\mathcal{X}_q$ leading to a set of q leakages $\mathcal{L}_q$. As a result of the attack, he obtains N_p lists of probabilities $\Pr[k_i^* | \mathcal{X}_q, \mathcal{L}_q]$, where $i \in [1 : N_p]$ and k_i^* denotes a subkey candidate among the $N_k = 2^b$ possible ones. Note that TA and LR-based attacks indeed output such probabilities directly. For other (typically non-profiled) attacks such as DPA or CPA, a Bayesian extension can be used for this purpose [17].

2.2 Rank Estimation

Concretely, each of the N_p lists of probabilities obtained by the divide-and-conquer adversary is typically small (i.e. easy to enumerate). So one can straightforwardly compute the rank of each subkey. The rank estimation problem is simply defined as the problem of estimating the master key rank based on the N_p lists $\Pr[k_i^* | \mathcal{X}_q, \mathcal{L}_q]$. Quite naturally, the problem is trivial when the attack is directly successful (i.e. when the master key is rated first). But it becomes tricky whenever this rank becomes larger. The solution in [18] was to organize the keys by sorting their subkeys according to the posterior probabilities provided by DPA, and to represent them as a high-dimensional dataspace (with N_p dimensions). The full key space can then be partitioned in two volumes: one defined by the key candidates with probability higher than the correct key, one defined by the key candidates with probability lower than the correct key. Using this geometrical representation, the rank estimation problem can be stated as the one of finding bounds for these “higher” and “lower” volumes. It essentially works by carving volumes representing key candidates on each side of their boundary, in order to progressively refine the (lower and upper) bounds on the key rank. As mentioned in introduction, this approach is efficient as long as the carved volumes are large enough, and becomes computationally intensive afterwards.

3 Simpler and More Efficient Rank Estimation

3.1 Algorithm Specification

We denote the DPA outcomes as lists of log probabilities $LP_i = \log(\Pr[k_i^* | \mathcal{X}_q, \mathcal{L}_q])$, and the histograms (with N_{bin} equally-sized bins) corresponding to these lists as $H_i = \text{hist}(LP_i, \text{bins})$, where bins is the (same) set of bins used for all histograms. We further denote the convolution between two histograms as $\text{conv}(H_i, H_j)$. From these notations, our rank estimation proposal is specified by Algorithm 1.

The algorithm exploits the property that for two multisets of numbers $\mathcal{S}_1$ and $\mathcal{S}_2$ of which the distribution is described by the histograms H_1, H_2 , the distribution of the numbers in the multiset $\mathcal{S} = \mathcal{S}_1 + \mathcal{S}_2 := \{x_1 + x_2 | x_1 \in \mathcal{S}_1, x_2 \in \mathcal{S}_2\}$ can be approximated by a convolution of the histograms H_1 and H_2 , *if the histograms use the same binsize*. That is if the histograms H_1 and H_2 describe the frequencies of numbers in the multisets $\mathcal{S}_1$ and $\mathcal{S}_2$, such that $H_1(x)$ gives the number of elements in $\mathcal{S}_1$ that equal x and $H_2(y)$ gives the number of elements in $\mathcal{S}_2$ that equal y , then the frequencies of numbers in the multiset $\mathcal{S}$ of all possible sums is approximated by the histogram H with $H(z) \approx \sum_{x+y=z} H_1(x) \times H_2(y) = \sum_x H_1(x) \times H_2(z-x)$. If $\mathcal{S}_1$ and $\mathcal{S}_2$ contain only integers, this is the convolution of the histograms H_1 and H_2 (considered as vectors). Similarly, if there is no quantization error (i.e. the log probabilities exactly equal the mid values of their respective bins), this property holds as well, provided that the bin widths of both histograms are equal. Thus, assuming that the quantization error are small

Algorithm 1. Rank estimation.**Input:** The key log probability $\log(\Pr[k|\mathcal{X}_q, \mathcal{L}_q])$ and the histograms H_i .**Output:** An approximation of k 's rank.*initialization:* $H_{\text{curr}} = H_1$;*histograms convolution:***for** $i = 2 : N_p$ $H_{\text{curr}} = \text{conv}(H_{\text{curr}}, H_i)$;**end***rank estimation:*

$$\text{estimated_rank} \approx \sum_{i=\text{bins}(\log(\Pr[k|\mathcal{X}_q, \mathcal{L}_q]))}^{N_p \cdot N_{\text{bin}} - (N_p - 1)} H_{\text{curr}}(i).$$

enough, the histogram of all possible sums of two log probability lists can be approximated by the convolution of the two corresponding histograms.

Note that the (log) probability of the correct key has to be known by the evaluator – as in [18]. Note also that the number of bins of the current histogram H_{curr} increases linearly with the number of convolutions executed N_p . Overall, the accuracy of the approximated rank essentially depends on the number of bins N_{bin} and number of pieces N_p , leading to the simple tightness vs. time complexity tradeoff discussed in the next sections. And the memory complexity is roughly $N_p \times N_{\text{bin}}$, which is easily manageable in practice (see Sect. 4.1).

3.2 Bounding the Error

Let us assume two log probabilities $LP_1^{(j)}$ and $LP_2^{(j)}$ corresponding to the j th candidates in the lists LP_1 and LP_2 . They are associated with two bins of central value $m_1^{(j)}$ and $m_2^{(j)}$ in the histograms H_1 and H_2 . Whenever summing those log probabilities (as required to combine two lists of probabilities), it may happen that the central value of the bin corresponding to $LP_1^{(j)} + LP_2^{(j)}$ is different than $m_1^{(j)} + m_2^{(j)}$ (which corresponds to the approximated sum of log probabilities obtained from the convolution in Algorithm 1). This typically occurs if the distance between the log probabilities $LP_1^{(j)}, LP_2^{(j)}$ and their bins' central values $m_1^{(j)}, m_2^{(j)}$ is too large, as illustrated by the following numerical example.

Example 1. Take two lists $LP_1 = \{0, 0.02, 0.07, 0.11, 0.14, 0.16, 0.19, 0.3\}$ and $LP_2 = \{0.02, 0.02, 0.036, 0.04, 0.12, 0.19, 0.24, 0.29\}$. For $N_{\text{bin}} = 3$, it leads to a common binsize of $S_{\text{bin}} = 0.1$, and central values $\{0.05, 0.15, 0.25\}$. Hence, we obtain $H_1 = \{3, 4, 1\}$ and $H_2 = \{4, 2, 2\}$. The convolution $H_3 = \text{conv}(H_1, H_2)$ is a histogram with $N_{\text{bin}} = 5$ and central values $\{0.1, 0.2, 0.3, 0.4, 0.5\}$, given by $H_3 = \{12, 22, 18, 10, 2\}$. As a result, the sum of log probabilities $LP_1^{(7)} + LP_2^{(8)}$ equals $0.19 + 0.29 = 0.48$ and should be placed in the bin with central value 0.5.

Yet, since their corresponding central values are 0.15 and 0.25, the convolution approximates their sum within the bin of central value $0.15 + 0.25 = 0.4$.

In other words, the rank estimation accuracy is limited by quantization errors (of one bin in our example). Fortunately, we can bound the number of bins between the result of the convolution and the real sum of log probabilities as follows.

Proposition 1. *Let $\{LP_i\}_{i=1}^{N_p}$ be N_p lists of log probabilities with their j th elements denoted as $LP_i^{(j)}$ and set in the bins of central values $m_i^{(j)}$ of the corresponding histograms $\{H_i\}_{i=1}^{N_p}$. The quantization error (in bins) between $\sum_{i=1}^{N_p} LP_i^{(j)}$ (i.e. the actual sum of log probabilities) and $\sum_{i=1}^{N_p} m_i^{(j)}$ (i.e. the sum of the bins' central values corresponding to these log probabilities) is at most $N_p/2$.*

Proof. If S_{bin} is the binsize, the equation $|LP_i^{(j)} - m_i^{(j)}| \leq \frac{S_{\text{bin}}}{2}$ holds for each $i \in [1 : N_p]$. Hence, by summing over all the pieces, we obtain:

$$-\frac{S_{\text{bin}}}{2} \times N_p \leq \sum_{i=1}^{N_p} (LP_i^{(j)} - m_i^{(j)}) \leq \frac{S_{\text{bin}}}{2} \times N_p.$$

Hence, we also have:

$$\left| \sum_{i=1}^{N_p} LP_i^{(j)} - \sum_{i=1}^{N_p} m_i^{(j)} \right| \leq \frac{N_p}{2} \times S_{\text{bin}},$$

which limits the distance between $\sum_{i=1}^{N_p} LP_i^{(j)}$ and $\sum_{i=1}^{N_p} m_i^{(j)}$ to $\frac{N_p}{2}$ bins. $\square$

Following, we can directly bound the estimated rank in Algorithm 1 with:

$$\text{rank_lower_bound} = \sum_{i=\text{bin}(\log(\Pr[k|\mathcal{X}_q, \mathcal{L}_q])) + N_p}^{N_p \cdot N_{\text{bin}} - (N_p - 1)} H_{\text{curr}}(i),$$

and:

$$\text{rank_upper_bound} = \sum_{i=\text{bin}(\log(\Pr[k|\mathcal{X}_q, \mathcal{L}_q])) - N_p}^{N_p \cdot N_{\text{bin}} - (N_p - 1)} H_{\text{curr}}(i),$$

where the N_p (vs. $N_p/2$) value comes from the fact that the distance limit holds for each list of log probabilities independently. Hence, a triangle inequality with $\sum_{i=1}^{N_p} m_i^{(j)}$ as origin gives us an interval of size $2 \times N_p$ bins around $\sum_{i=1}^{N_p} LP_i^{(j)}$.

4 Performance Evaluation

In this section, we analyze the performance of Algorithm 1. For comparison purposes, we first use the same AES experimental setup as Veyrat-Charvillon et al. We then extend our experiments to larger key sizes. In the latter case, we only run our new algorithm as comparisons can only become more favorable in this context. Indeed, the carving phase of the proposal in [18] is quite sensitive to an increase of the number of dimensions. In order to keep their algorithm (time) efficient, the authors therefore start their rank estimation by merging some dimensions, which is quite memory intensive. Note that the functional correctness of our algorithm derives from the previous section. Yet, we tested its implementation by comparing our results with the ones obtained by enumeration for key ranks up to 2^{32} , and made sure that these results were consistent with the ones obtained using the open source code of the Eurocrypt 2013 paper.

4.1 AES-128 Case Study

As in [18], we considered simulated attacks where the adversary is provided with 16 leakage samples of the shape $l_i = \text{HW}(\text{S}(x_i \oplus k_i)) + n_i$ for $i \in [1 : 16]$, where HW is the Hamming weight function, S is the AES S-box, k_i and x_i are the previously defined subkeys and corresponding plaintext bytes, and n_i is a Gaussian-distributed random noise. We then performed classical TAs using the noise variance and number of plaintexts as parameters, so that the adversary computes 16 lists of 256 posterior probabilities. As in the previous paper as well, the efficiency of the rank estimation algorithms was quite independent of the type of leakage exploited: the only influencing factor in our performance evaluations was the rank of the correct key candidate. For this purpose, we started by reproducing an experiment where we launched many independent attacks, with different security levels and increasing time complexities, and plotted the resulting bounds' tightness (defined in Footnote 2). The left (resp. right) part of Fig. 1 contains the results of this experiment for the Eurocrypt 2013 algorithm³ (resp. Algorithm 1). In both cases, they were obtained on a desktop computer with an Intel i7 core, without any parallelization effort (details on the implementation of Algorithm 1 are in Appendix B). Two clear observations can be extracted from this figure. First, the security levels leading to the most complex rank estimations differ for the two algorithms (i.e. key ranks around 2^{80} are most challenging with the Eurocrypt 2013 algorithm, enumerable key ranks are the most challenging with ours). Second and most importantly, the new bounds are much tighter (less than one bit of distance between the bounds) and obtained much faster (in less than a second). Note that the experiments with 0.05 s, 0.5 s and 5 s of computations in the right part of the figure respectively correspond to 5K, 50K and 500K starting bins in the initial histograms. The latter case corresponds to 64 MB of memory using doubling point precision.

³ Using 8 lists of size 2^{16} for illustration.

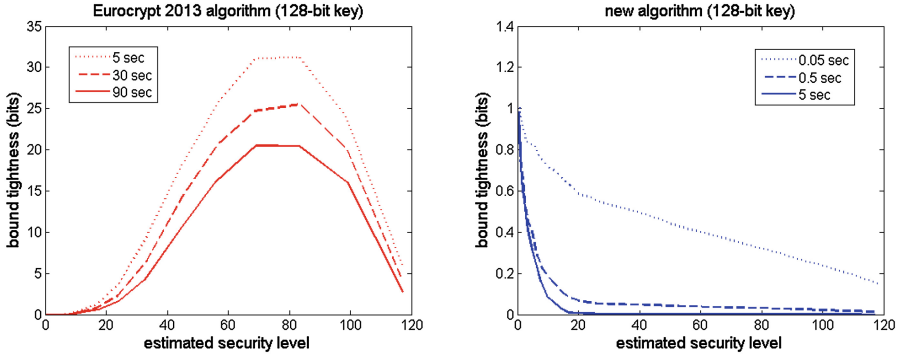


Fig. 1. Rank estimation tightness in function of the security level.

In order to make the comparison even more explicit, we additionally provide the “convergence graphs” where the upper and lower bounds on the key rank are plotted in function of the time complexity. As clear from Fig. 2, the convergence is incomparably faster with the histogram-based approach than with the Eurocrypt 2013 one. Additional results for other relevant security levels (namely ≈ 60 -bit and ≈ 100 -bit) are provided in Appendix, Figs. 5 and 6. For completeness, we also provide a zoom of Fig. 2 (right) and its companion where the X axis is expressed in number of starting bins in Appendix, Fig. 7.

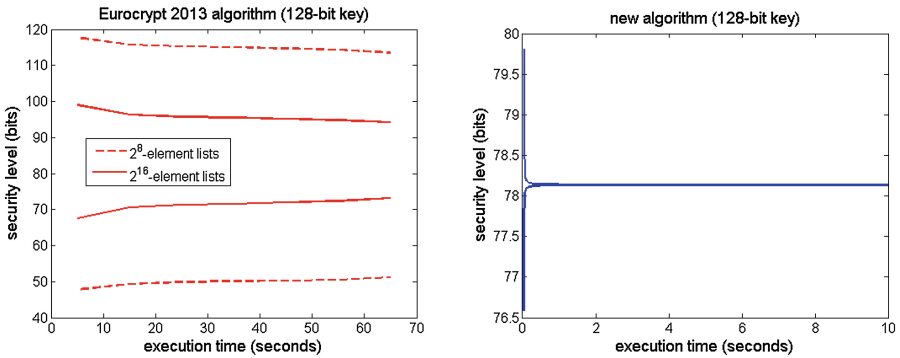


Fig. 2. Rank estimation convergence for an ≈ 80 -bit security level.

4.2 Larger Key Sizes

In order to analyze situations with larger key sizes, we simply extended our AES simulated setting to more 8-bit pieces. Namely, we considered key sizes of 256, 512 and 1024 bits (i.e. $N_p = 32, 64, 128$). We omit the figure corresponding to the 256-bit case because it is extremely close to the 128-bit one, and represent

the convergence graphs of the two latter cases in Fig. 3. While the application of the Eurocrypt 2013 method hardly provides useful results on this context, the figure clearly exhibits that Algorithm 1 produces tight bounds within seconds of computation, even in this challenging case. As expected, the increase of execution time in the 1024-bit example mainly corresponds to the convolutions' cost that becomes significant as the number of bins increases (in $N_{\text{bin}} \log(N_{\text{bin}})$). This trends remains observed as long as the memory requirements of Algorithm 1 can fit in RAM, which was always the case in our experiments.

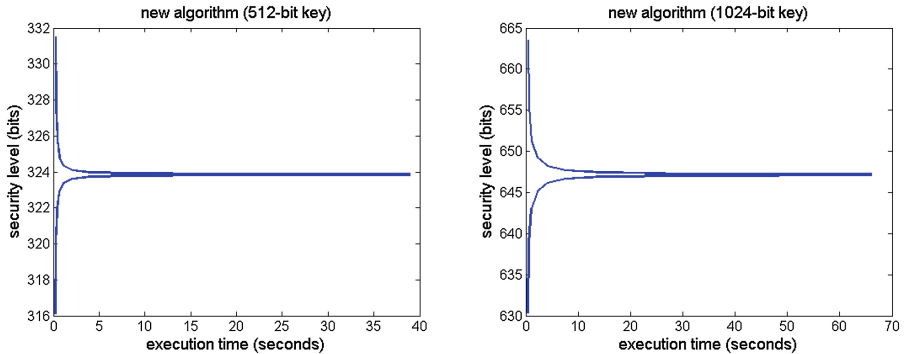


Fig. 3. Rank estimation convergence for 512- and 1024-bit keys.

Eventually, we additionally provide graphs representing the bounds' tightness in function of the security level for these 512- and 1024-bit cases in Fig. 4. They essentially confirm the observation already made in Fig. 1 that the most challenging key ranks to estimate (with our new algorithm) are the lower ones. Note that in these latter cases, the experiments with 0.1 s (resp. 0.5) and 1 (resp. 5) were performed with respectively 2 K and 20K starting bins.

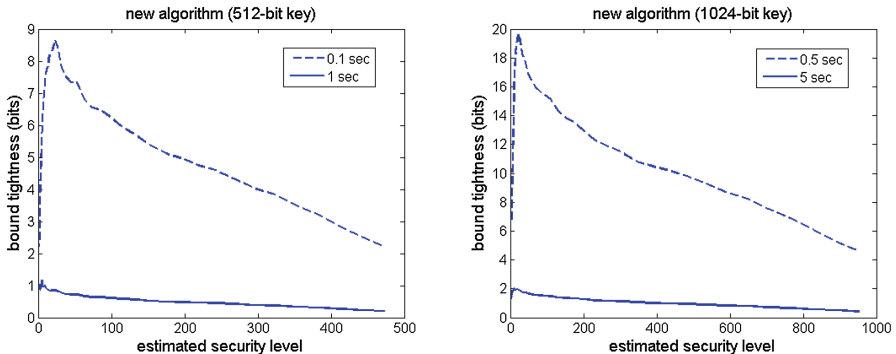


Fig. 4. Rank estimation tightness in function of the security level.

5 Conclusions

This paper provides a surprisingly simple alternative of rank estimation algorithm, that significantly outperforms the previous proposal from Eurocrypt 2013. It has natural applications in the field of side-channel cryptanalysis and is a tool of choice for evaluation laboratories willing to quantify the security level of a leaking implementation in a rigorous manner. More generally, it can also be useful in the evaluation of any cryptanalytic technique where the advantage gained is not sufficient for key recovery and not predictable by analytical means.

Acknowledgements. F.-X. Standaert is a research associate of the Belgian Fund for Scientific Research (FNRS-F.R.S.). This work has been funded in parts by the European Commission through the ERC project 280141 (CRASH).

A Additional Figures

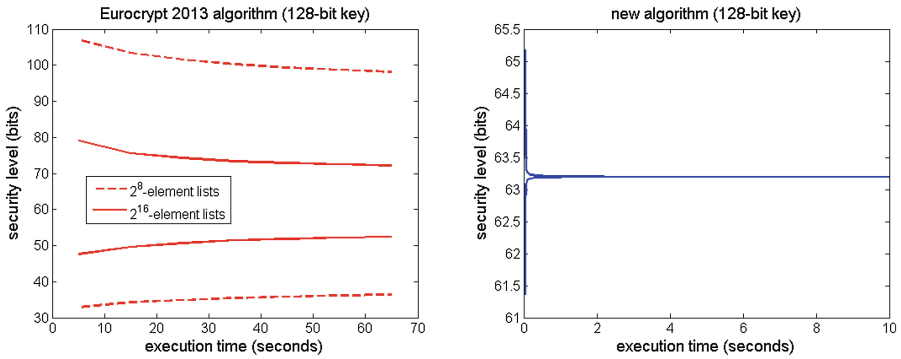


Fig. 5. Rank estimation convergence for an ≈ 60 -bit security level.

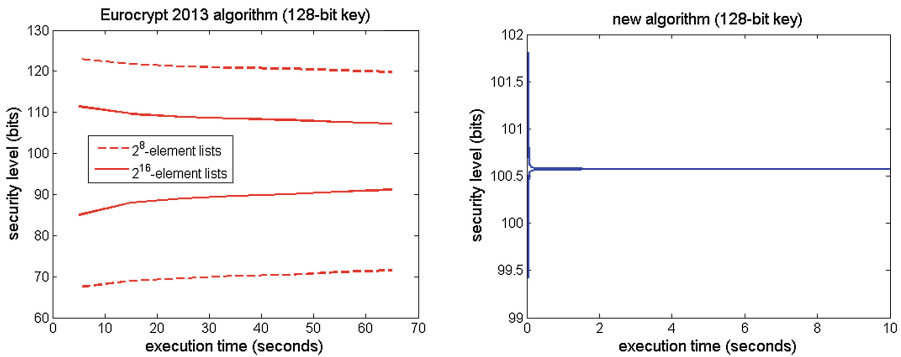


Fig. 6. Rank estimation convergence for an ≈ 100 -bit security level.

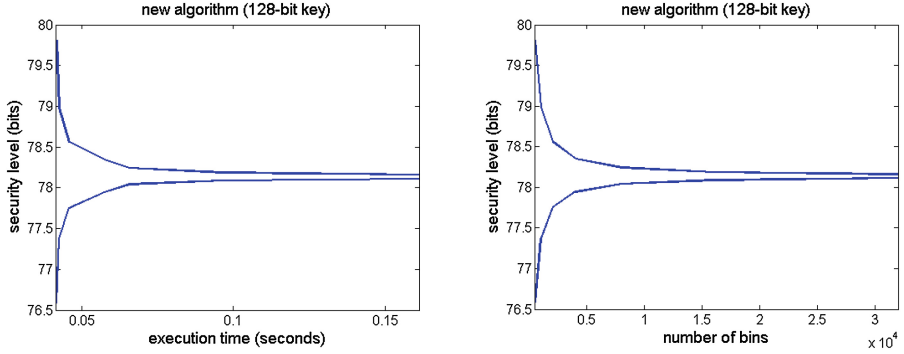


Fig. 7. Rank estimation convergence for an ≈ 80 -bit security level (zoom).

B Implementation Details

One additional advantage of Algorithm 1 is that it is straightforward to implement, in particular if efficient convolution algorithms for vectors of arbitrary precision integers are available out of the box, as in a number of mathematical programming languages. Our experiments were performed with Matlab scripts, which turn out to be sufficiently efficient for all the case studies we considered. Yet, we mention that if efficient convolutions algorithms are not available out of the box, they can easily be implemented using more readily available primitives. One possible approach is to use a mixture of floating point arithmetic and representation of large numbers according to the Chinese Remainder Theorem. For example, a set of moderately sized primes, like the 20 largest primes below 10000, is chosen, and each histogram is converted into a CRT representation by using the 20 integer vectors that are obtained by modular reduction with each of the 20 chosen primes. This particular choice of primes is suitable to represent numbers of up to 265 bits (i.e. for rank estimation of 256-bit keys). In this example, when two histograms are to be convoluted, 20 regular convolutions are computed, each one modulo the corresponding prime base. To speed up the computation, each of these 20 convolutions can be performed by multiplication in Fourier space with regular double precision floating point arithmetic, i.e. two FFTs, one element-wise multiplication of two complex vectors, and one inverse FFT. Since the exact result is known to consist of integer values, computational inaccuracies can unambiguously be removed by rounding to the nearest integers. In the CRT representation, the input values are bounded by the prime bases, and this sufficiently limits the requirements on floating point precision. After rounding to integer values, modular reductions to the corresponding prime bases are performed to obtain the CRT representation of the convolution result. The result can be left in CRT representation until all histograms (i.e. the histograms for all subkeys) have been convoluted. Only a single CRT back transform to large integers is required after all the histograms have been convoluted.

References

1. Brier, E., Clavier, C., Olivier, F.: Correlation power analysis with a leakage model. In: Joye, M., Quisquater, J.-J. (eds.) CHES 2004. LNCS, vol. 3156, pp. 16–29. Springer, Heidelberg (2004)
2. Chari, S., Jutla, C.S., Rao, J.R., Rohatgi, P.: Towards sound approaches to counteract power-analysis attacks. In: Wiener [20], pp. 398–412
3. Chari, S., Rao, J.R., Rohatgi, P.: Template attacks. In: Kaliski Jr., B.S., Koç, Ç.K., Paar, C. (eds.) Cryptographic Hardware and Embedded Systems - CHES 2002. LNCS, vol. 2523, pp. 13–28. Springer, Heidelberg (2002)
4. Duc, A., Faust, S., Standaert, F.-X.: Making masking security proofs concrete. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015. LNCS, vol. 9056, pp. 401–429. Springer, Heidelberg (2015)
5. Dziembowski, S., Pietrzak, K.: Leakage-resilient cryptography. In: 49th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2008, 25–28 October 2008, Philadelphia, PA, USA, pp. 293–302. IEEE Computer Society (2008)
6. Gierlichs, B., Batina, L., Tuyls, P., Preneel, B.: Mutual information analysis. In: Oswald, E., Rohatgi, P. (eds.) CHES 2008. LNCS, vol. 5154, pp. 426–442. Springer, Heidelberg (2008)
7. Herbst, C., Oswald, E., Mangard, S.: An AES smart card implementation resistant to power analysis attacks. In: Zhou, J., Yung, M., Bao, F. (eds.) ACNS 2006. LNCS, vol. 3989, pp. 239–252. Springer, Heidelberg (2006)
8. Kocher, P.C., Jaffe, J., Jun, B.: Differential power analysis. In: Wiener [20], pp. 388–397
9. Lange, T., van Vredendaal, C., Wakker, M.: Kangaroos in side-channel attacks. In: Joye, M., Moradi, A. (eds.) CARDIS 2014. LNCS, vol. 8968, pp. 104–121. Springer, Heidelberg (2015)
10. Mangard, S., Oswald, E., Standaert, F.-X.: One for all - all for one: unifying standard differential power analysis attacks. IET Inf. Secur. **5**(2), 100–110 (2011)
11. Mangard, S., Popp, T., Gammel, B.M.: Side-channel leakage of masked CMOS gates. In: Menezes, A. (ed.) CT-RSA 2005. LNCS, vol. 3376, pp. 351–365. Springer, Heidelberg (2005)
12. Pan, J., van Woudenberg, J.G.J., den Hartog, J.I., Witteman, M.F.: Improving DPA by peak distribution analysis. In: Biryukov, A., Gong, G., Stinson, D.R. (eds.) SAC 2010. LNCS, vol. 6544, pp. 241–261. Springer, Heidelberg (2011)
13. Rivain, M., Prouff, E.: Provably secure higher-order masking of AES. In: Mangard, S., Standaert, F.-X. (eds.) CHES 2010. LNCS, vol. 6225, pp. 413–427. Springer, Heidelberg (2010)
14. Schindler, W., Lemke, K., Paar, C.: A stochastic model for differential side channel cryptanalysis. In: Rao, J.R., Sunar, B. (eds.) CHES 2005. LNCS, vol. 3659, pp. 30–46. Springer, Heidelberg (2005)
15. Standaert, F.-X., Malkin, T.G., Yung, M.: A unified framework for the analysis of side-channel key recovery attacks. In: Joux, A. (ed.) EUROCRYPT 2009. LNCS, vol. 5479, pp. 443–461. Springer, Heidelberg (2009)
16. Standaert, F.-X., Pereira, O., Yu, Y.: Leakage-resilient symmetric cryptography under empirically verifiable assumptions. In: Canetti, R., Garay, J.A. (eds.) CRYPTO 2013, Part I. LNCS, vol. 8042, pp. 335–352. Springer, Heidelberg (2013)
17. Veyrat-Charvillon, N., Gérard, B., Renauld, M., Standaert, F.-X.: An optimal key enumeration algorithm and its application to side-channel attacks. In: Knudsen, L.R., Wu, H. (eds.) SAC 2012. LNCS, vol. 7707, pp. 390–406. Springer, Heidelberg (2013)

18. Veyrat-Charvillon, N., Gérard, B., Standaert, F.-X.: Security evaluations beyond computing power. In: Johansson, T., Nguyen, P.Q. (eds.) EUROCRYPT 2013. LNCS, vol. 7881, pp. 126–141. Springer, Heidelberg (2013)
19. Veyrat-Charvillon, N., Medwed, M., Kerckhof, S., Standaert, F.-X.: Shuffling against side-channel attacks: a comprehensive study with cautionary note. In: Wang, X., Sako, K. (eds.) ASIACRYPT 2012. LNCS, vol. 7658, pp. 740–757. Springer, Heidelberg (2012)
20. Wiener, M.J. (ed.): CRYPTO '99. LNCS, vol. 1666. Springer, Heidelberg (1999)
21. Ye, X., Eisenbarth, T., Martin, W.: Bounded, yet sufficient? How to determine whether limited side channel information enables key recovery. In: Joye, M., Moradi, A. (eds.) CARDIS 2014. LNCS, vol. 8968, pp. 215–232. Springer, Heidelberg (2015)